ROBOT OPERATING SYSTEM

Isha Pradeep Bhat
Department of Computer Science and Engineering
Manipal Institute of Technology
Manipal, India
isha.mitmpl2026@learner.manipal.edu

Abstract— The ROS 2 is an open source middleware framework widely used for developing modular, distributed, and scalable robotic applications. It provides communication mechanisms, development tools, package management, and standardized interfaces that allow independent software components to exchange information efficiently. This report presents the fundamental concepts of ROS 2 Humble, including the computation graph, nodes, topics, services, actions, parameters, Quality of Service (QoS), custom interfaces, workspaces, and the Colcon build system.

Keywords— ROS2 computation graph, nodes, topics, services, actions, parameters, custom interfaces, colcon and workspace structure, and QoS settings

INTRODUCTION

Robotic applications commonly combine sensing, perception, estimation, planning, and control. Implementing all functions in one large program can make development difficult to test, extend, and maintain. ROS 2 addresses this challenge by providing a middleware framework in which functionality is divided into independent nodes that exchange typed data through standard communication interfaces.

ROS 2 Humble Hawksbill is intended for Ubuntu 22.04 LTS in the installation path used in this report. The assignment applies these concepts to two examples: a wheel-speed calculation pipeline and a three-node palindrome-processing pipeline.

For Windows users, Ubuntu 22.04 can be installed through Windows Subsystem for Linux (WSL) or through dual boot using a bootable USB drive. Dual boot provides a native Ubuntu environment, while WSL provides an integrated Linux environment within Windows. Graphical applications and hardware access may require additional configuration depending on the selected method.

The implementation consists of two practical tasks. The Task One consists of creating a custom message called TaskOne which transmits the angular velocity and wheel radius from publisher to subscriber. The subscriber calculates the longitudinal wheel speed based on the angular velocity and radius being used. In the Task Two, three ROS 2 nodes have been developed that will accept the user's strings, check if each string is a palindrome, send the result as a Boolean integer, and send a text message back. Thus, both implementations illustrate the inter-node communications between the nodes, definitions of the custom messages, organization of the packages, and the use of ROS 2 command-line tools.

ROS 2 COMPUTATION GRAPH

The ROS 2 computation graph is a logical view of the active nodes and their communication relationships. Nodes expose publishers, subscriptions, services, actions, and parameters. Topics connect publishers to compatible subscriptions; services connect clients and servers; actions connect clients and servers through goal, feedback, and result communication. The graph is a runtime view, not a complete representation of every internal software dependency.

A graph makes modularity visible and supports debugging. If a subscriber receives no data, the graph and CLI can help determine whether the publisher is running, whether the topic name is correct, and whether compatible endpoints have been discovered. A visible edge is useful evidence of a communication relationship, but message values and application correctness should also be verified using terminal output and topic inspection.

WORKSPACE

A workspace is a directory that contains the packages used to develop, build, and run a robotic application. It provides an organized environment for managing source code, dependencies, custom message definitions, and other project resources. A typical ROS 2 workspace contains a src directory for source packages, while the build, install, and log directories are generated during the build process using Colcon.

NODES

A node is an individual executable program that performs a specific task within a robotic system. Nodes are the fundamental building blocks of the ROS 2 computation graph, and each node is designed to carry out a particular function, such as reading sensor data, processing information, controlling motors, or communicating with other components. Nodes can exchange data using topics, request and receive responses through services, and

perform long-running tasks using actions. They can also use parameters to configure their behaviour without modifying the source code.

ros2 node list displays the node names currently discovered in the ROS graph.

ros2 node info gives information about a selected node, including its publishers, subscribers, services, and actions where available.

TOPICS

topics are communication channels that allow nodes to exchange data using a publish-subscribe mechanism. A node that sends data is called a publisher, while a node that receives data is called a subscriber. Topics support continuous data exchange, making them suitable for applications that require frequent updates. Each topic has a name and uses a specified message type to define the structure of the data being transmitted. Multiple nodes can publish to or subscribe to the same topic, enabling efficient communication between different components of a robotic system. Thus, topics provide a flexible and efficient way to enable asynchronous communication between ROS 2 nodes.

ros2 topic list
ros2 topic list -t
ros2 topic info <topic_name>
ros2 topic info <topic_name> --verbose

SERVICES

Services provide a communication mechanism that allows one node to send a request to another node and receive a response. They follow a request-response model, making them suitable for operations that require a specific result rather than continuous data transmission. A service consists of two parts: a request, which contains the information sent by the client node, and a response, which contains the result returned by the server node. Each service is defined by a service interface that specifies the data types used in the request and response

ros2 service list
ros2 service list -t
ros2 service type <service_name>
ros2 service find <service_type>

Inspecting and calling a service
ros2 interface show <service_type>
ros2 service call <service_name> <service_type> '<request_data>'

ACTIONS

Actions are a communication mechanism used to perform long-running tasks that require continuous feedback and a final result. They follow a goal-feedback-result model, in which a client node sends a goal to a server node, the server provides feedback while executing the task, and a final result is returned when the task is completed. Actions also support goal cancellation, allowing a task to be stopped before completion.

PARAMETER

parameters are configuration values associated with a node that allow its behaviour to be adjusted without modifying or recompiling the source code. Parameters can store different types of data, including integers, floating-point numbers, strings, and Boolean values. They are commonly used to configure settings such as sensor thresholds, control gains, operating modes, and vehicle speed limits. Each parameter belongs to a specific node and can be declared, accessed, modified, and inspected using ROS 2 APIs and command-line tools.

QUALITY OF SERVICE

Quality of Service (QoS) refers to a set of policies that control how data is transmitted between nodes through topics, ensuring that communication meets the requirements of a robotic application. QoS settings allow developers to manage aspects such as message reliability, data history, message durability, and timing constraints.

Policy	Meaning	Typical consideration
Reliability	Reliable or best effort	Reliable attempts delivery; best effort prioritizes timely delivery and may lose samples.
Durability	Volatile or transient local	Transient local can retain samples for late-joining compatible subscribers.
History	Keep last or keep all	Controls the sample history retained by middleware, subject to resources.
Depth	Queue depth for keep-last history	A larger depth can buffer bursts but consumes resources and may preserve stale data.
Deadline	Expected maximum interval between samples	Useful for detecting missed expected publication intervals.
Lifespan	Maximum useful age of a message	Helps prevent delivery of expired data.
Liveliness	How publisher activity is asserted or detected	Can help identify a publisher that is no longer active.

MESSAGES

In ROS 2, messages are data structures used to exchange information between nodes through topics. They define the format and type of data that a node publishes or receives, allowing different components of a robotic system to communicate in a standardized way. ROS 2 provides built-in message types such as String for text, Int32 for integer values, Float64 for decimal numbers, and Twist for representing linear and angular velocity.

ros2 topic echo <topic_name>

The echo command creates a subscriber and prints messages it receives. It is useful for confirming whether a
publisher is active and whether the values look correct.

PACKAGE

A package is an organizational unit that contains ROS 2 code, resources, configuration files, and information about dependencies. It helps developers organise their programs so they can be built, installed, run, shared, and reused easily.

Python package (ament_python)

- package.xml - contains package information and dependency declarations.
- setup.py - defines how the Python package is installed and which executables it provides.
- setup.cfg - configures installation settings for executables where required.
- resource/ - contains the package marker file.
- Python module directory - contains the Python source files and usually __init__.py.

Example of a package will be turtlesim. The turtlesim package provides a simple simulated turtle for learning ROS concepts. The turtlesim node displays the turtle, while a keyboard teleoperation node reads key presses and publishes movement commands.

ros2 run <package_name> <executable_name>

REMAPPING

Remapping changes a ROS name, such as a topic name or node name, at launch time. It can let an existing node communicate using a different topic without changing its source code. Remapping affects the running instance's name resolution; it does not rewrite the original program.

ros2 run <package_name> <executable_name> --ros-args -r old_topic:=new_topic

The exact remapping names depend on the node's interfaces. Remapping is especially useful when running multiple copies of a node or connecting components that use different topic names.

SOURCING THE ROS INSTALLTION

Sourcing a setup script configures the current terminal for ROS 2. It sets environment variables so the shell can locate ROS commands, packages, libraries, and related resources. If the environment is not sourced, a terminal may not recognise ROS commands or may not discover the intended packages.

source /opt/ros/humble/setup.bash

Each new terminal is a separate shell environment, so it may need the ROS setup script sourced again unless
sourcing is configured automatically in the shell startup file.

UNDERLAY AND OVERLAY

The **underlay** is an existing ROS installation or workspace that provides packages. An **overlay** is another workspace sourced on top of it, usually containing the developer's own packages or modified versions of packages. This layered approach allows a developer to extend an installation without replacing the entire ROS distribution.

ROS2 COMMANDS

Command	What it tells you
ros2 node list	Which nodes are currently discovered
ros2 node info	Interfaces exposed by a selected node
ros2 topic list -t	Topic names and message types
ros2 topic echo	Messages arriving on a topic
ros2 topic info --verbose	Publisher/subscriber endpoint and QoS details
ros2 service list -t	Available service names and types
ros2 action list -t	Available action names and types
ros2 interface show	Fields in a message, service, or action definition
ros2 param list	Parameters exposed by nodes
ros2 pkg list	Packages discoverable in the environment

TASK 1

The objective of Question 1 was to implement communication between two ROS 2 nodes using a custom message interface. The publisher collects angular velocity and radius values and transmits them to a subscriber, which calculates the longitudinal speed.

A custom message package named taskone_msgs was created using the ament_cmake build system. The message file TaskOne.msg contains two fields:

- float64 ang_vel
- float64 radius

These fields represent angular velocity and radius, respectively. The interface package generates the message type used by the publisher and subscriber.

The publisher node was implemented in Python using rclpy. It obtains angular velocity and radius values from the user, stores them in a TaskOne message, and publishes the message on the /taskone topic. The publisher and subscriber use the same custom message type, enabling structured data exchange.

The subscriber node listens to the /taskone topic. Whenever it receives a message, it extracts the angular velocity and radius and calculates longitudinal speed using: v=wr. where (v) is longitudinal speed, (\omega) is angular velocity, and (r) is radius. The computed speed is displayed in the termina

The publisher and subscriber were executed successfully. For an angular velocity of 10 rad/s and a radius of 2 m, the subscriber calculated a longitudinal speed of 20 m/s. The rqt_graph visualization was used to inspect the nodes and their topic connection, confirming the communication structure of the implementation.



TASK 2

The objective of Question 2 was to implement a multi-node ROS 2 application in which one node publishes a user-provided string, a second node checks whether the string is a palindrome, and a third node converts the result into a textual response and sends it back to the checking node.

A custom interface package named rt2_msgs was created using ament_cmake. The message file RT2String.msg contains the field string data, which stores the input text. This custom interface enables the publisher and palindrome-checking node to exchange string data through a ROS 2 topic

The first node, string_publisher, obtains a string from the user and publishes it repeatedly on the /rt2_string topic using the custom RT2String message. A timer is used to publish the stored string at regular intervals.

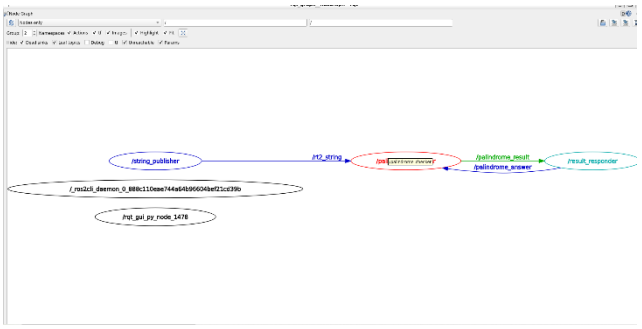
The second node, palindrome_checker, subscribes to /rt2_string and checks whether the received string reads identically forwards and backwards. If the string is a palindrome, the node publishes the integer 1; otherwise, it publishes 0. The result is sent on the /palindrome_result topic using the standard Int32 message type. The checker also subscribes to /palindrome_answer to receive the textual response from the third node.

The third node, result_responder, subscribes to /palindrome_result. It converts the integer result into a standard string message: 1 becomes Yes, while 0 becomes No. The response is published on /palindrome_answer, allowing the palindrome-checking node to receive and display the final answer.

The three nodes were executed successfully and their communication was inspected using rqt_graph. For the input AABAA, the checker produced the result 1, and the responder returned Yes. The implementation demonstrates communication between multiple nodes using custom and standard message interfaces, with separate topics for input, processing results, and the final response.

CONCLUSION

This taskphase provided practical experience with the fundamental concepts of ROS 2, including workspaces, packages, nodes, topics, custom message interfaces, publishers, subscribers, and computation graph visualization. Question 1 demonstrated communication between two nodes using a custom message to calculate longitudinal speed. Question 2 extended this approach to a three-node system that performs palindrome checking and returns a textual response. The implementations improved understanding of modular robotics software development and demonstrated how independent ROS 2 nodes can cooperate through structured message exchange.



REFERNCE

https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html
https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html
https://www.geeksforgeeks.org/techtips/setup-dual-boot-with-linux-and-windows
https://www.youtube.com/watch?si=BK8k4o9eczblK9Xi&v=F3n0SMAFheM&feature=youtu.be

.